

Trabajo Práctico 4

1. Responder breve y claramente:

a. ¿Por qué en la complejidad espacial se utilizan MT con una cinta de entrada de sólo lectura?

En la C.E se usan MT con una cinta de entrada readonly porque permite hacer un análisis de cuánta memoria (celdas de la/s cinta/s) se necesita para resolver un problema independiente del tamaño de la cadena de entrada.

Si no se hiciera esto, el espacio mínimo para cualquier problema sería el lineal ($O(n)$, por el tamaño de la entrada) lo cual está mal porque hay problemas que se pueden resolver en espacio sublineal, como los de *LOGSPACE*.

b. ¿Por qué si una MT tarda tiempo $poly(n)$ entonces ocupa espacio $poly(n)$, y si ocupa espacio $poly(n)$ puede llegar a tardar tiempo $exp(n)$?

1. MT tarda tiempo polinomial $\rightarrow$ ocupa espacio polinomial:

- En cada paso o unidad de tiempo la MT puede moverse a la izquierda, moverse a la derecha, o quedarse en el mismo lugar.
- Si tenemos $poly(n)$ pasos, entonces la MT no puede moverse más de $poly(n)$ celdas a la derecha, ni más de $poly(n)$ celdas a la izquierda, porque si pudiera esto implicaría que la MT hace más de una cosa a la vez por cada unidad de tiempo, lo cual sabemos es imposible.
- Por lo tanto la MT ocupa como máximo $poly(n)$ celdas, es decir, espacio polinomial.

2. MT ocupa espacio polinomial $\rightarrow$ puede tardar tiempo exponencial:

- Espacio $S(n)$ implica tiempo $O(c^{S(n)})$ para alguna constante c .
- Si una MT ocupa espacio $poly(n)$, entonces el tiempo que puede tardar es $O(c^{poly(n)}) = O(exp(n))$ por propiedades de las funciones exponenciales y polinomiales.
- Por lo tanto, si una MT ocupa espacio polinomial, puede llegar a tardar tiempo exponencial.

c. ¿Por qué los lenguajes de la clase *LOGSPACE* son tratables?

Los lenguajes de *LOGSPACE* (lenguajes decididos por una MT en espacio logarítmico) son tratables por la implicación fundamental de que toda MT que trabaja en espacio logarítmico **necesariamente** se ejecuta en tiempo polinomial.

Espacio $S(n)$ implica tiempo $O(c^{S(n)})$ para alguna constante c . Si una MT ocupa espacio logarítmico, entonces tarda tiempo $O(c^{log(n)})$. Como $c^{log(n)} = n^{log(c)}$ por propiedades de los logaritmos, entonces $O(c^{log(n)}) = O(n^k)$ para alguna constante k . Por lo tanto, toda MT que trabaja en espacio logarítmico se ejecuta en tiempo polinomial, lo cual implica que los lenguajes de *LOGSPACE* son tratables.

2. Describir la idea general de una MT M que:

a. Decida el lenguaje $L = \{a^n b^n \mid n \geq 1\}$ en espacio logarítmico. Ayuda: basarse en el ejemplo mostrado en la clase 7.

Primeras cadenas del lenguaje: $ab, aabb, aaabbb, aaaabbbb, \dots$

Se construye una MT M con una cinta de entrada de sólo lectura y cuatro cintas de trabajo:

1. Recibe una cadena w en la cinta de entrada.
2. Hace $i = 1$ en la cinta de trabajo 1.
3. Hace $j = n$ en la cinta de trabajo 2, con $n = |w|$. Si n es impar, M rechaza.
4. Copia el símbolo i de w a la cinta de trabajo 3.
5. Copia el símbolo j de w a la cinta de trabajo 4.
6. Si no se cumple que el símbolo de la cinta de trabajo 3 es a y el símbolo de la cinta de trabajo 4 es b , entonces M rechaza.
7. Hace $i = i + 1$ en la cinta de trabajo 1.
8. Hace $j = j - 1$ en la cinta de trabajo 2.
9. Una vez $i > j$, M acepta.
10. Vuelve al paso 3.

L pertenece a $LOGSPACE$ porque:

1. Las cintas de trabajo 1 y 2 almacenan un número hasta n en binario, lo cual requiere $O(\log(n))$ celdas.
2. Las cintas de trabajo 3 y 4 almacenan un símbolo, lo cual es espacio constante $O(1)$.
3. Total: $O(\log(n)) + O(\log(n)) + O(1) + O(1) = O(\log(n))$.
4. Por lo tanto $L \in LOGSPACE$.

b. Decida el lenguaje SAT en espacio polinomial. Ayuda: la generación y la evaluación de una asignación de valores de verdad se pueden efectuar en tiempo polinomial.

Se construye una MT M con una cinta de entrada de sólo lectura y dos cintas de trabajo:

1. Recibe una fórmula booleana ϕ en la cinta de entrada tal que n es la cantidad de variables de $\phi(x_1, x_2, \dots, x_n)$.
2. Escribe n ceros en la cinta de trabajo 1, representando la asignación de valores de verdad $x_1 = 0, x_2 = 0, \dots, x_n = 0$, donde el número cero representa el valor de verdad **falso**.
3. Evalúa ϕ en la cinta de trabajo 2 usando la asignación de valores de verdad representada en la cinta de trabajo 1 junto con los operadores lógicos presentes en ϕ .
4. Si el resultado de la evaluación es un 1 (V), entonces M acepta porque la fórmula es satisfactible. Si el resultado es un 0 (F), pasa al paso 5.
5. Genera la siguiente asignación de valores de verdad posible en la cinta de trabajo 1, lo que implica sumar 1 en binario a la asignación actual. Si hay overflow, M rechaza. Si no, limpia la cinta de trabajo 2 y vuelve al paso 3.

SAT pertenece a $PSPACE$ porque:

1. La cinta de trabajo 1 almacena n bits por lo cual el espacio es $O(n)$.
2. La cinta de trabajo 2 realiza la evaluación de una fórmula booleana, lo cual se puede hacer en espacio proporcional al tamaño de la misma, es decir $O(|\phi|)$.
3. Total: $O(n) + O(|\phi|) = O(poly(n))$.
4. Por lo tanto $SAT \in PSPACE$.

3. Probar que $NP \subseteq PSPACE$ de dos maneras:

a. Usando que todos los lenguajes de NP se reducen polinomialmente a SAT .

Ayuda: tener en cuenta el resultado del ejercicio 2.b.

1. Hipótesis:
 - i. Se sabe que $SAT \in NPC$, por lo cual $SAT \in NP$ y todos los lenguajes de NP se reducen polinomialmente a SAT .
 - ii. Se sabe que $SAT \in PSPACE$.
2. Como todos los lenguajes de NP se reducen polinomialmente a SAT , entonces para cada lenguaje $L \in NP$ existe una función de reducción polinomial f tal que para toda cadena w :
 - i. Si $w \in L$, entonces $f(w) \in SAT$.
 - ii. Si $w \notin L$, entonces $f(w) \notin SAT$.
3. Para decidir si una cadena w pertenece a L , se construye una MT M que hace lo siguiente:
 - i. Dada una cadena w , calcula $f(w)$ en tiempo polinomial.
 - ii. Luego, decide si $f(w) \in SAT$ usando la MT que decide SAT en espacio polinomial.
 - iii. Si $f(w) \in SAT$, entonces M acepta porque $w \in L$. Si $f(w) \notin SAT$, entonces M rechaza porque $w \notin L$.
4. Por lo tanto, M decide L en espacio polinomial, lo cual implica que $L \in PSPACE$.

b. Usando que si un lenguaje L pertenece a NP , entonces existe una MT M_1 capaz de verificar en tiempo $poly(|w|)$ si una cadena w pertenece a L , con la ayuda de un certificado x de tamaño $poly(|w|)$. Ayuda: con dicha hipótesis, probar que existe una MT M_2 que decide L en espacio $poly(|w|)$ sin la ayuda de ningún certificado.

1. Hipótesis:
 - i. Se sabe que si un lenguaje L pertenece a NP , entonces existe una MT M_1 capaz de verificar en tiempo $poly(|w|)$ si $w \in L$, con la ayuda de un certificado x de tamaño $poly(|w|)$.
2. Para decidir si una cadena w pertenece a L , se construye una MT M_2 que hace lo siguiente:
 - i. Dada una cadena w , M_2 genera todas las posibles cadenas x de tamaño $poly(|w|)$, lo cual se puede hacer en espacio polinomial porque se reusa el espacio de trabajo en cada generación.
 - ii. Para cada cadena x generada, M_2 simula la MT M_1 con la entrada w y el certificado x .

- iii. Si para alguna cadena x , la MT M_1 acepta, entonces M_2 acepta porque $w \in L$. Si para todas las cadenas x , la MT M_1 rechaza, entonces M_2 rechaza porque $w \notin L$.
3. Por lo tanto, M_2 decide L en espacio polinomial, lo cual implica que $L \in PSPACE$.

4. Supongamos que existe una MT M de tiempo polinomial que, dado un grafo G , devuelve un circuito de Hamilton de G si existe o responde no si no existe. Describir la idea general de una MT M' que, utilizando M , decida en tiempo polinomial si un grafo G tiene un circuito de Hamilton. Ayuda: basarse en el ejemplo mostrado en la clase 7 con $FSAT$ y SAT .

...

5. Probar que si se puede encontrar en tiempo $poly(n)$ un circuito de Hamilton mínimo de un grafo completo ponderado G , entonces también se puede decidir en tiempo $poly(n)$ si G tiene un circuito de Hamilton cuyos arcos suman $\leq B$.

...